

AFC Data Engineering Project

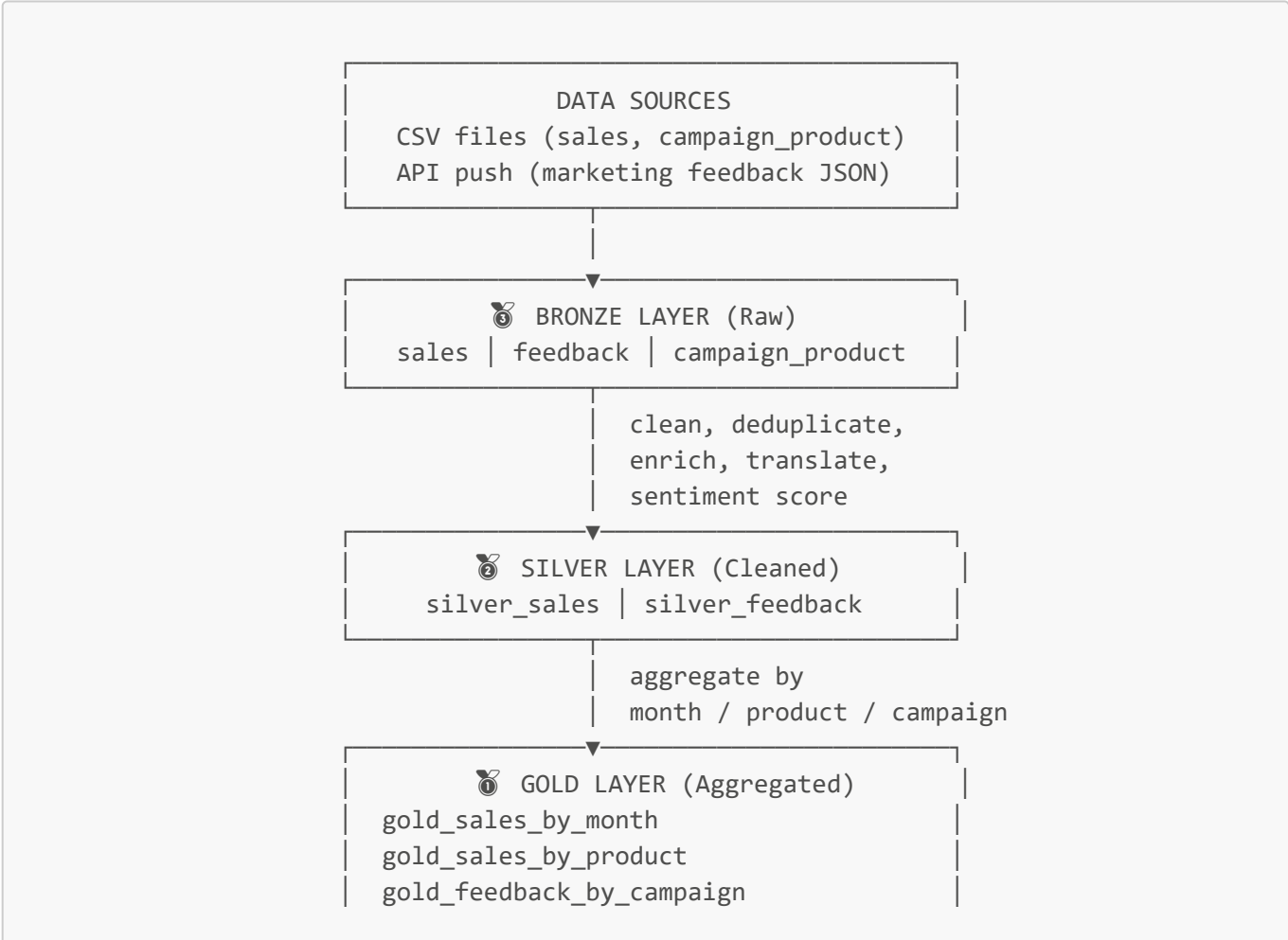
Nugget Data & AI Initiative (N.D.A.I) — Armoric Fried Chicken

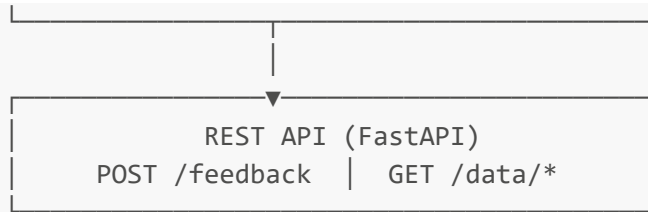
A data engineering pipeline that ingests sales and marketing feedback data, processes it through a Bronze/Silver/Gold medallion architecture, applies multilingual sentiment analysis, and exposes the results via a REST API.

Table of Contents

- [Architecture Overview](#)
- [Project Structure](#)
- [Prerequisites](#)
- [Installation](#)
- [Configuration](#)
- [Running the Project](#)
- [API Reference](#)
- [Testing the Pipeline](#)
- [Database Access](#)

Architecture Overview





Sentiment Analysis: feedback comments are automatically detected for language, translated to English if needed (via Google Translate), then scored with VADER. This supports multilingual feedback from all AFC markets.

Project Structure

```

afc-data-engineering-project/
├── .env                    # Environment variables (DB credentials)
├── requirements.txt        # Python dependencies
├── create_tables.py        # Standalone table creation script
├── run_pipeline.py         # Main pipeline orchestrator
├── data/
│   ├── raw/
│   │   ├── csv/           # Drop sales.csv and campaign_product.csv here
│   │   └── feedbacks/     # Feedback JSON files stored here by the API
├── docker/
│   ├── .env               # Docker environment variables
│   └── docker-compose.yml # PostgreSQL + Metabase container definitions
├── src/
│   ├── api/
│   │   └── main.py         # FastAPI application (POST + GET endpoints)
│   ├── db/
│   │   ├── connection.py  # Database connection helper
│   │   └── schema.py      # Table definitions (Bronze/Silver/Gold)
│   ├── ingestion/
│   │   ├── load_csv.py    # Loads sales and campaign_product CSV files
│   │   └── load_feedback_json.py # Loads feedback JSON into Bronze
│   ├── storage/
│   │   └── json_writer.py  # Writes incoming API feedback to JSON file
│   └── transformation/
│       ├── silver.py      # Bronze → Silver (clean, enrich, sentiment)
│       └── gold.py        # Silver → Gold (aggregate for dashboards)
  
```

Prerequisites

- Python 3.11+
- Docker Desktop

- Git

Installation

1. Clone the repository

```
git clone https://github.com/luna-vyer/afc-data-engineering-project
cd afc-data-engineering-project
```

2. Install Python dependencies

```
pip install -r requirements.txt
```

3. Create your `.env` file at the project root (see [Configuration](#))

4. Start PostgreSQL and Metabase

```
docker-compose -f docker/docker-compose.yml up -d
```

5. Verify the containers are running

```
docker ps
# You should see afc_postgres running on port 5432
# You should see afc_metabase running on port 3000
```

6. Create the Metabase internal database

```
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "CREATE DATABASE
metabase;"
```

Metabase takes ~60 seconds to initialize on first launch.

Configuration

Create a `.env` file at the project root with the following variables:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=afc_db
```

```
DB_USER=afc_user
DB_PASSWORD=afc_super_password
```

These values must match the credentials in [docker/.env](#):

```
POSTGRES_USER=afc_user
POSTGRES_PASSWORD=afc_super_password
POSTGRES_DB=afc_db
```

Running the Project

Step 1 — Add input data

Place your CSV files in [data/raw/csv/](#):

- [sales.csv](#) — columns: `username`, `sale_date`, `country`, `product`, `quantity`, `unit_price`, `total_amount`
- [campaign_product.csv](#) — columns: `campaign_id`, `product`

Sample data can be generated using [api_pusher](#).

Step 2 — Start the API server

```
uvicorn src.api.main:app --reload
```

The API will be available at <http://localhost:8000>. Interactive documentation: <http://localhost:8000/docs>

Step 3 — Push feedback data (optional)

Use [api_pusher](#) to push feedback via the API. In [config.ini](#), set:

```
endpoint_url = http://localhost:8000/feedback
```

Or send a request manually:

```
curl -X POST http://localhost:8000/feedback \  
-H "Content-Type: application/json" \  
-d '{"username": "user_1", "feedback_date": "2026-02-28", "campaign_id":  
"CAMP001", "comment": "Great chicken!"}'
```

Step 4 — Access the dashboards

Open **http://localhost:3000** in your browser to access Metabase.

Two dashboards are available:

- **Sales Dashboard** — revenue by product, country, and over time
- **Marketing & Sentiment Dashboard** — feedback sentiment scores, by product and campaign

Connect Metabase to the database using:

- Host: `postgres`
- Port: `5432`
- Database: `afc_db`
- User: `afc_user`
- Password: `afc_super_password`

Step 5 — Run the pipeline

```
python run_pipeline.py
```

This will execute all layers in order:

 Creating tables...

 Bronze – ingesting raw data...

 Silver – cleaning, enriching and scoring sentiment...

 Gold – aggregating for dashboards...

 Pipeline finished successfully

API Reference

Full interactive docs available at **http://localhost:8000/docs**

Method	Endpoint	Description
POST	<code>/feedback</code>	Ingest one or multiple feedback records
GET	<code>/data/sales</code>	Browse raw sales data (Bronze)
GET	<code>/data/feedback</code>	Browse raw feedback data (Bronze)
GET	<code>/data/campaign-products</code>	Browse campaign/product mapping (Bronze)
GET	<code>/data/silver/sales</code>	Cleaned sales data (Silver)
GET	<code>/data/silver/feedback</code>	Cleaned feedback with sentiment scores (Silver)
GET	<code>/data/gold/sales-by-month</code>	Revenue aggregated by month/country/product (Gold)
GET	<code>/data/gold/sales-by-product</code>	Overall product performance (Gold)

Method	Endpoint	Description
GET	/data/gold/feedback-by-campaign	Feedback volume and sentiment by campaign (Gold)
GET	/data/ingestion-log	Pipeline run history and status

All GET endpoints support `?limit=50&offset=0` for pagination. `/data/gold/sales-by-month` also supports `?country=France&product=Fried+Wings` filters.

Testing the Pipeline

Automated testing with api_pusher

Clone and configure [api_pusher](#):

```
git clone https://github.com/Prjprj/api_pusher
cd api_pusher
```

Edit `config.ini`:

```
endpoint_url = http://localhost:8000/feedback
```

Run the pusher, then execute the pipeline:

```
python run_pipeline.py
```

Manual verification

Check row counts after the pipeline:

```
# Bronze
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "SELECT COUNT(*) FROM sales;"
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "SELECT COUNT(*) FROM feedback;"

# Silver
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "SELECT COUNT(*) FROM silver_sales;"
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "SELECT comment, sentiment_score, sentiment_label FROM silver_feedback;"

# Gold
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "SELECT * FROM
```

```
gold_sales_by_product ORDER BY total_revenue DESC;"

# Pipeline log
docker exec -it afc_postgres psql -U afc_user -d afc_db -c "SELECT * FROM
ingestion_log ORDER BY ingested_at DESC LIMIT 10;"
```

Database Access

Connect to the database directly using psql:

```
docker exec -it afc_postgres psql -U afc_user -d afc_db
```

Or use any PostgreSQL client (DBeaver, pgAdmin, TablePlus) with:

```
Host:      localhost
Port:      5432
Database:  afc_db
User:      afc_user
Password:  afc_super_password
```

Tables

Layer	Table	Description
Bronze	sales	Raw sales transactions
Bronze	feedback	Raw customer feedback
Bronze	campaign_product	Campaign to product mapping
Bronze	ingestion_log	Pipeline run history
Silver	silver_sales	Cleaned, deduplicated sales
Silver	silver_feedback	Enriched feedback with sentiment scores
Gold	gold_sales_by_month	Revenue aggregated by month/country/product
Gold	gold_sales_by_product	Overall product performance
Gold	gold_feedback_by_campaign	Sentiment aggregated by campaign